



Group 2

Browser Navigation: Two Stacks และ ArrayList

Member

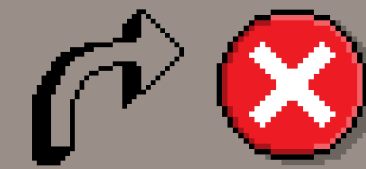
นายธีรวิรุฒ ภูมิมาลา 67122250024

นายธีรภัทร์ นิลทรัพย์ 68122250005

นางสาววงศ์ศุลี เอี่ยมพริ้ง 68122250034

นายโชติพันธ์ สีแพน 68122250091

นายพรพล ยิ่งเจริญ 68122250086



Case Story: Browser Navigation

ระบบ Browser ต้องจัดการประวัติการเปิดเว็บไซต์ของผู้ใช้ โดยผู้ใช้สามารถเปิดเว็บไซต์ใหม่ ย้อนกลับไปยังหน้าก่อนหน้า (Back) และกลับไปยังหน้าที่เคยเข้าชม (Forward) ได้

ปัญหาที่ต้องจัดการคือ เมื่อผู้ใช้กด Back แล้วเปิดเว็บไซต์ใหม่ ประวัติ Forward ที่มีอยู่จะต้องถูกล้าง เพื่อให้การทำงานเหมือน Browser จริง

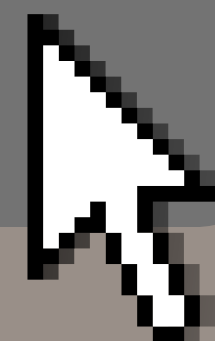
ตัวอย่างสถานการณ์:

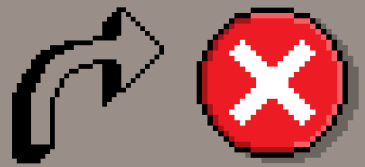
Visit A → Visit B → Visit C → BACK → B → VISIT D

เมื่อเปิด D หลังจากกด Back แล้ว หน้า C จะถูกล้างออกจาก Forward History

เป้าหมาย:

ออกแบบโครงสร้างข้อมูลที่สามารถจัดการ Back / Forward / Visit ได้อย่างถูกต้องและมีประสิทธิภาพ





Input, Output และ Constraints

Input

- ข้อมูลหน้าเว็บไซต์ Page
- Page ID
- Page Title
- URL
- Visited Time
- คำสั่งของ Browser
- VISIT
- BACK
- FORWARD

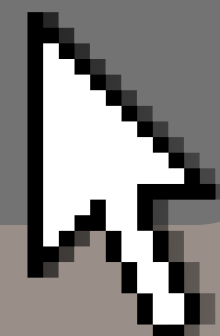
Output

- แสดงหน้าปัจจุบัน (Current Page)
- แสดง Back History
- แสดง Forward History
- แสดงผลการทำงานของแต่ละคำสั่ง

Constraints

- หากไม่มีหน้าก่อนหน้า → ไม่สามารถ BACK
- หากไม่มีหน้าถัดไป → ไม่สามารถ FORWARD
- เมื่อ VISIT หน้าใหม่หลังจาก BACK → ต้องล้าง Forward History

โค้ดกำหนดข้อมูลของแต่ละหน้าไว้ในคลาส Page และเก็บ visitedTime ด้วย LocalDateTime





Algorithm A: Two-Stack Method

ใช้โครงสร้างข้อมูล 3 ส่วน

backStack → เก็บหน้าที่สามารถย้อนกลับได้

forwardStack → เก็บหน้าที่สามารถไปข้างหน้าได้

currentPage → หน้าปัจจุบัน

การทำงาน

VISIT

1. นำ currentPage ใส่ backStack
2. เปลี่ยน currentPage เป็นหน้าใหม่
3. ล้าง forwardStack

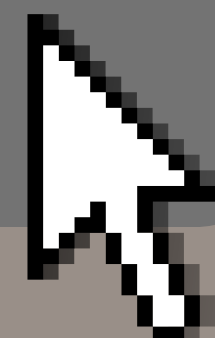
BACK

1. นำหน้าปัจจุบันใส่ forwardStack
2. pop หน้าจาก backStack มาเป็นหน้าปัจจุบัน

FORWARD

1. นำหน้าปัจจุบันใส่ backStack
2. pop หน้าจาก forwardStack มาเป็นหน้าปัจจุบัน

โครงสร้างนี้ถูก implement ในคลาส TwoStackBrowser โดยใช้
Deque<Page>





Algorithm B: ArrayList + Current Index

ใช้ตัวแปรหลัก 2 ตัว

- historyList → เก็บประวัติหน้าเว็บทั้งหมด
- currentIndex → ระบุดำแหน่งหน้าปัจจุบัน

การทำงาน

VISIT

1. ตรวจสอบว่ามี Forward History หรือไม่
2. ถ้ามี → ลบข้อมูลหลัง currentIndex
3. เพิ่มหน้าใหม่เข้า historyList
4. เพิ่ม currentIndex

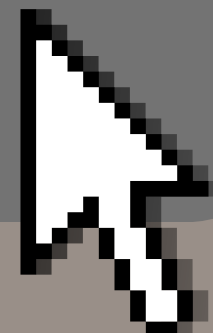
BACK

ลด currentIndex ลง 1

FORWARD

เพิ่ม currentIndex ขึ้น 1

วิธีนี้ถูก implement ในคลาส ArrayListBrowser โดยใช้ ArrayList และ currentIndex



ตัวอย่าง Step-by-step (Algorithm A: Two-Stack Method)

ลำดับการทำงาน

0. INIT

Back: [] ว่าง
Current: null
Forward: [] ว่าง
อธิบายการทำงาน: สถานะ
เริ่มต้นระบบ

1. VISIT A

Back: []
Current: A
Forward: []
อธิบายการทำงาน: เปิดหน้าแรก

2. VISIT B

Back: [A]
Current: B
Forward: []
อธิบายการทำงาน: นำ A ลง
Back Stack, เปิด B

3. VISIT C

Back: [A, B]
Current: C
Forward: []
อธิบายการทำงาน: นำ B ลง
Back Stack, เปิด C

4. BACK

Back: [A]
Current: B
Forward: [C]
อธิบายการทำงาน: นำ C ลง
Forward Stack, ดึง B จาก
Back

5. BACK

Back: []
Current: A
Forward: [B, C]
อธิบายการทำงาน: นำ B ลง
Forward Stack, ดึง A จาก
Back

6. FORWARD

Back: [A]
Current: B
Forward: [C]
อธิบายการทำงาน: นำ A ลง
Back Stack, ดึง B จาก
Forward

7. VISIT D

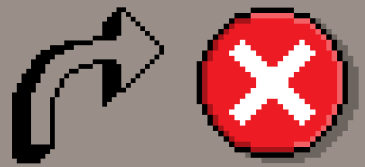
Back: [A, B]
Current: D
Forward: [] (ล้างทิ้ง)
อธิบายการทำงาน: นำ B ลง
Back, เปิด D, **ล้าง C ออกจาก
Forward**

8. BACK

Back: [A]
Current: B
Forward: [D]
อธิบายการทำงาน: นำ D ลง Forward Stack,
ดึง B จาก Back

9. FORWARD

Back: [A,B]
Current: D
Forward: []
อธิบายการทำงาน: นำ B ลง Back Stack, ดึง D
จาก Forward



Correctness Argument

ทำไม Algorithm จึงทำงานถูกต้อง?

กรณี VISIT

- หน้าปัจจุบันถูกเก็บไว้ใน Back History
- เปลี่ยนไปยังหน้าใหม่
- Forward History ถูกล้าง

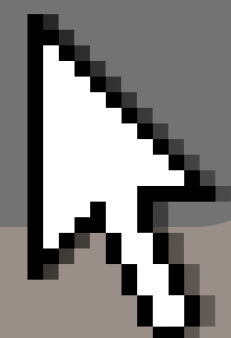
กรณี FORWARD

- ถ้ามี Forward History → ย้ายหน้าปัจจุบันไป Back
- นำหน้าล่าสุดจาก Forward มาเป็น Current

กรณี BACK

- ถ้ามี Back History → ย้ายหน้าปัจจุบันไป Forward
- นำหน้าล่าสุดจาก Back มาเป็น Current

ดังนั้นทั้งสอง Algorithm สามารถรักษาความสัมพันธ์ของ
Back History → Current Page → Forward History ได้อย่างถูกต้อง





Big-O Analysis

N = จำนวนหน้าใน History
F = จำนวนหน้าใน Forward History

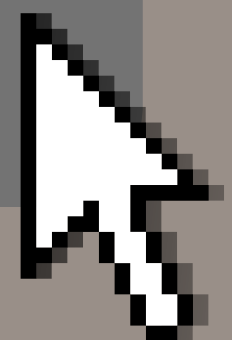
จุดสำคัญ

เมื่อ VISIT หลังจาก BACK

- Two-Stack ต้อง clear() Forward Stack
- ArrayList ต้องลบข้อมูลหลัง currentIndex

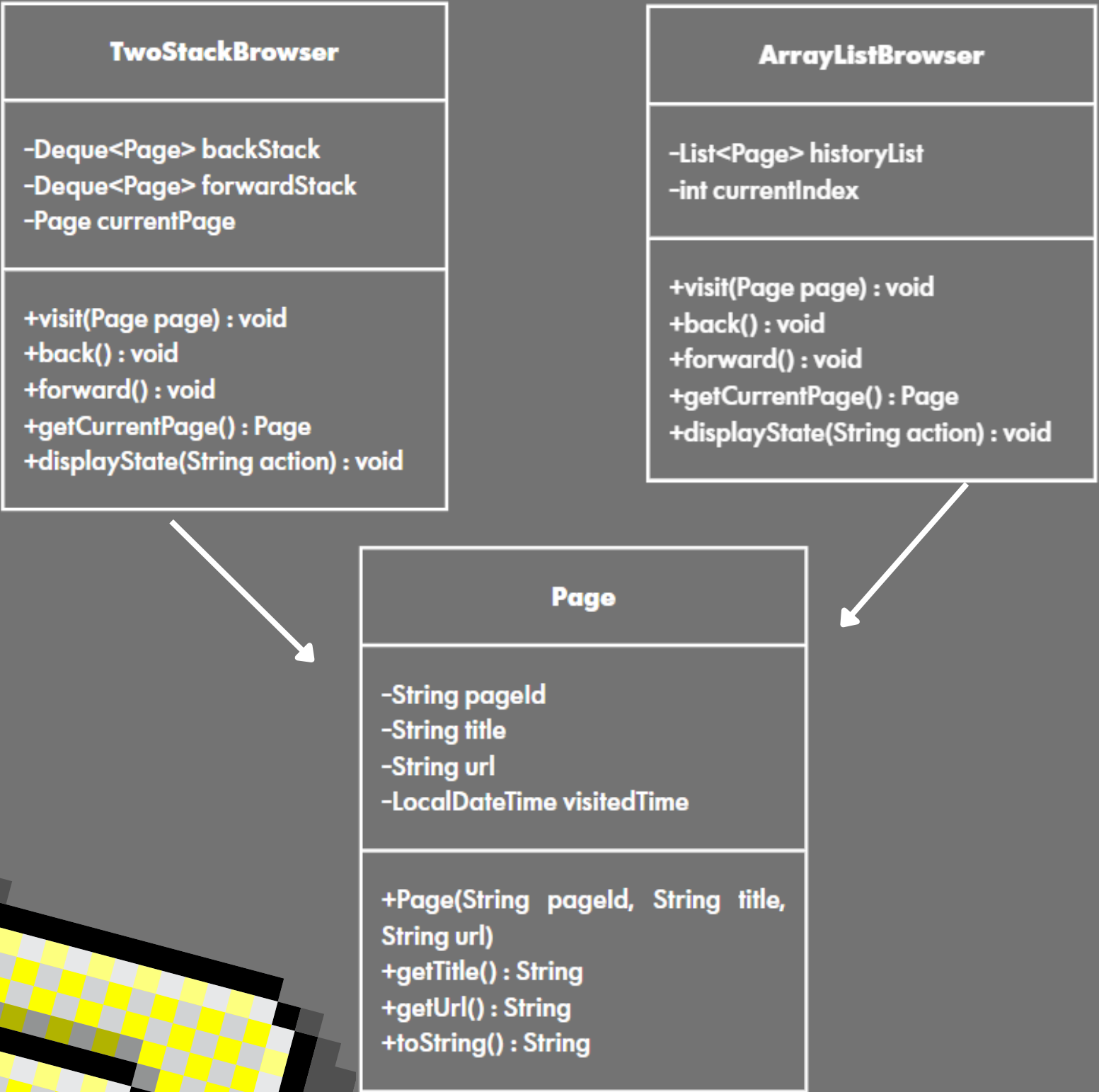
ดังนั้นกรณีที่มี Forward History จำนวนมาก การ VISIT อาจใช้เวลา $O(N)$ ในกรณีแย่ที่สุด

Operation	Two-Stack	ArrayList + Index
BACK	$O(1)$	$O(1)$
FORWARD	$O(1)$	$O(1)$
VISIT	$O(F)$ worst case	$O(F)$ worst case
CURRENT	$O(1)$	$O(1)$
Space	$O(N)$	$O(N)$



Class Diagram IIa: Java Implementation

Class Diagram



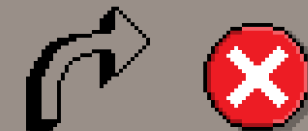
Java Implementation

Algorithm A

```
Deque<Page> backStack;  
Deque<Page> forwardStack;  
Page currentPage;
```

Algorithm B

```
List<Page> historyList;  
int currentIndex;
```

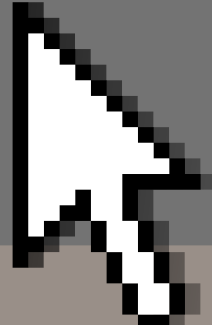


Test Cases

No.	Test Case	Input Operations	Expected Output
1	กด Back เมื่อไม่มีหน้าก่อนหน้า	INIT → BACK	แจ้งว่าไม่สามารถ Back ได้ และ Current Page ยังคงเป็น null
2	กด Forward เมื่อไม่มีหน้าถัดไป	VISIT A → FORWARD	แจ้งว่าไม่สามารถ Forward ได้ และ Current Page = A
3	เปิดหน้าแรก	VISIT A	Current = A, Back = [], Forward = []
4	เปิดหน้าเดิมซ้ำ	VISIT A → VISIT A	Current = A, Back = [A]
5	Back หลายครั้ง	VISIT A → B → C → BACK → BACK	Current = A, Back = [], Forward = [B, C]
6	Forward หลายครั้ง	จากข้อ 5 → FORWARD → FORWARD	Current = C, Back = [A, B], Forward = []
7	Back แล้วเปิดหน้าใหม่	VISIT A → B → C → BACK → VISIT D	Current = D, Back = [A, B], Forward = []
8	ประวัติต่อจำนวนมาก	VISIT 1 → 100,000 → BACK 50,000 → VISIT new	ระบบทำงานครบถ้วน ล้าง Forward 50,000 รายการ และไม่เกิด Error

จุดประสงค์ของการทดสอบ

- ตรวจสอบการทำงานของ BACK และ FORWARD
- ตรวจสอบกรณีที่ไม่มีประวัติให้ย้อนกลับหรือไปข้างหน้า
- ตรวจสอบการเปิดหน้าใหม่หลังจาก BACK
- ตรวจสอบการจัดการประวัติจำนวนมาก
- ตรวจสอบความถูกต้องและความเสถียรของระบบ



Experimental Results

ทดลองกับจำนวนหน้า

- 1,000 หน้า
- 10,000 หน้า
- 50,000 หน้า
- 100,000 หน้า

ผลที่สังเกตได้

- Two-Stack ใช้เวลาค่อนข้างคงที่ แม้จำนวนหน้าจะเพิ่มขึ้น
- ArrayList ใช้เวลามากขึ้นตามจำนวน Forward History ที่ต้องลบ
- เมื่อมีข้อมูล 100,000 หน้า ArrayList ใช้เวลาประมาณ 2.15 ms ขณะที่ Two-Stack ใช้เวลาประมาณ 0.001 ms
- ผลการทดลองแสดงให้เห็นว่า Two-Stack มีประสิทธิภาพดีกว่าในกรณีที่ต้องล้าง Forward History จำนวนมาก

ผลการทดลองเปรียบเทียบประสิทธิภาพ

จำนวนหน้า (N)	จำนวนที่ต้องล้าง (N/2)	Two-Stack Method	ArrayList Method
1,000	500	~1,200 ns (0.001 ms)	~18,500 ns (0.018 ms)
10,000	5,000	~1,300 ns (0.001 ms)	~142,000 ns (0.142 ms)
50,000	25,000	~1,400 ns (0.001 ms)	~890,000 ns (0.890 ms)
100,000	50,000	~1,500 ns (0.001 ms)	~2,150,000 ns (2.150 ms)

สรุปผล

เมื่อจำนวนหน้าเพิ่มขึ้น Two-Stack มีเวลาทำงานเพิ่มขึ้นเพียงเล็กน้อย ในขณะที่ ArrayList มีเวลาเพิ่มขึ้นอย่างชัดเจน จึงเหมาะสมกว่าสำหรับการจัดการ Back/Forward History ในกรณีนี้

สรุปและวิธีที่กลุ่มเลือก

สรุปผล

- ทั้ง Two-Stack และ ArrayList + Current Index สามารถจัดการ VISIT, BACK และ FORWARD ได้ถูกต้อง
- ทั้งสองวิธีสามารถจัดการกรณี BACK แล้ว VISIT หน้าใหม่ โดยล้าง Forward History ได้
- จากการทดลอง เมื่อจำนวนหน้าเพิ่มขึ้น Two-Stack ใช้เวลาค่อนข้างคงที่
- ส่วน ArrayList ใช้เวลามากขึ้น เมื่อจำนวน Forward History ที่ต้องลบเพิ่มขึ้น

วิธีที่กลุ่มเลือก: Two-Stack Method

เหตุผลที่เลือก

1. เหมาะกับการทำงานของ Browser ที่มี BACK และ FORWARD
2. การ BACK และ FORWARD ทำได้ใน $O(1)$
3. จากการทดลอง Two-Stack ใช้เวลาน้อยกว่า ArrayList อย่างชัดเจน
4. เมื่อมีประวัติจำนวนมาก ประสิทธิภาพของ Two-Stack มีความสม่ำเสมอว่า

ข้อสรุป

กลุ่มเลือก Two-Stack Method เพราะเหมาะสมกับลักษณะการทำงานของ Browser Navigation และจากผลการทดลองพบว่าใช้เวลาในการจัดการ Forward History น้อยกว่า ArrayList โดยเฉพาะเมื่อมีข้อมูลจำนวนมาก